
DemoEvolve: Overcoming Sparse Feedback in Agentic Harness Evolution with Demonstrations

Lirong Che^{1,2} Yuzhe Yang² Peiwen Lin²
 Chuang Wang² Xueqian Wang² Jian Su²
¹Tsinghua University ²AgiBot

Abstract

Agent harness evolution improves frozen language-model agents by modifying the executable structures around them. We study this paradigm as a form of sample-efficient fast adaptation: instead of updating model weights, an agent can acquire task-specific competence by changing its external harness, while leaving the base model’s general capabilities intact. Prior work shows that self-generated rollouts can support harness search, suggesting that agents may acquire new task competence through practice. Yet in long-horizon stochastic environments, self-practice becomes fragile: rewards are sparse, outcomes are high-variance, and failures are hard to attribute to concrete harness mechanisms. We introduce DemoEvolve, a demonstration-bootstrapped approach to harness evolution. When reward-only search is too broad and noisy, competent human trajectories serve as expert reference experience for the coding proposer, guiding harness-level diagnosis and editing. Experiments on Liar’s Dice show that self-rollout evolution can work when episodes are short and failures are attributable. In contrast, Balatro exposes a harder long-horizon stochastic regime, where self-rollout evolution is misled by sparse feedback and candidate-selection noise, while tutorial-like textual knowledge alone does not yield stable improvement. Under the same limited budget, DemoEvolve produces more effective and auditable harness edits and achieves better performance. Overall, demonstrations make sparse-feedback harness evolution more diagnosable, localizable, and stable.

1 Introduction

Large language model agents are shaped not only by model weights, but also by the external harnesses that surround them [38, 31, 33]. Recent work on harness evolution treats these structures as editable programs around a frozen model: a coding proposer inspects prior code, scores, and execution traces, then writes a new candidate harness [12, 42, 15, 17]. This suggests an exciting possibility: agent improvement can be cast as sample-efficient fast adaptation through automated program search over the external harness, adding task-specific competence without retraining the model or risking weight-space interference with its general capabilities.

Existing successes in harness evolution are strongest in text- and code-centric domains, where modern LLMs already possess substantial priors [13, 21, 15, 17]. We ask whether the same paradigm can support LLM-non-native task worlds, where improvement requires more than eliciting latent knowledge: the harness must help structure state, action, and feedback in an unfamiliar interactive environment. We therefore use games as a controlled research regime for this transition. Fixed seeds and explicit action interfaces make candidate comparisons reproducible under a controlled evaluation protocol, while stochastic events and delayed outcomes make them meaningfully stateful and interactive. Figure 1 frames turn-based games as an intermediate regime between model-native artifact optimization and open-ended real-time worlds.

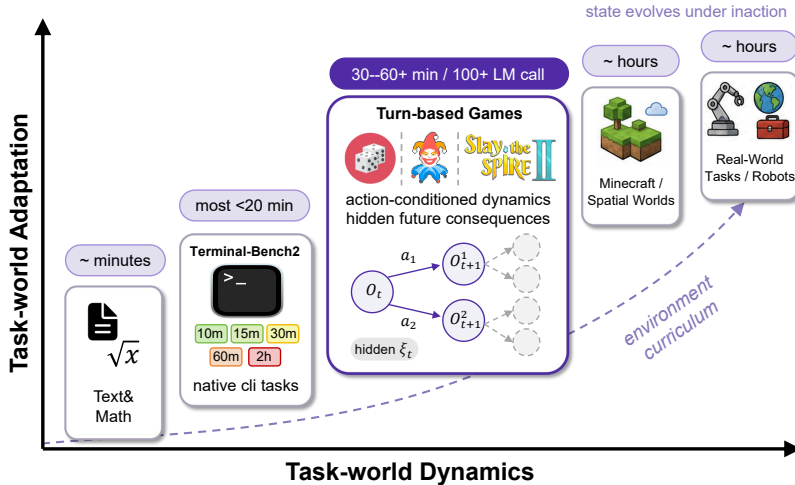


Figure 1: **Games as a controlled testbed for task-world adaptation.** We position domains by task-world dynamics and adaptation requirement. Turn-based games introduce action-conditioned, seed-dependent futures and sparse long-horizon feedback, but remain reproducible through fixed seeds and explicit action interfaces. They therefore test whether harness evolution can move beyond model-native priors toward task-specific state abstractions and strategies.

This intermediate regime stresses a core assumption behind harness evolution: prior experience must be diagnostic enough to guide the next program edit. In short-horizon or model-native tasks, this is often plausible because failures are relatively local and traces are short. In long-horizon interactive worlds, however, sparse outcomes can be delayed and misleading. A failed run may contain hundreds of state transitions, small early decisions can reshape the future state distribution, and stochasticity can make an inactive harness edit appear beneficial by chance. The challenge is therefore not only search over harness programs, but credit assignment: inferring which external mechanism should change from noisy trajectory-level feedback.

This paper studies this credit-assignment problem in LLM-non-native games. We use TextArena Liar’s Dice [8] as a short-horizon positive control, and Balatro [3, 2] as a long-horizon stochastic setting. Liar’s Dice shows that self-rollout harness evolution can work when failures are local and attributable. Balatro, by contrast, exposes a regime where sparse, high-variance feedback can make reward-only search unreliable: apparent score gains may not correspond to causal, model-facing harness improvements. This motivates demonstration-guided harness evolution as a way to make sparse feedback more diagnosable and localizable.

In summary, this paper makes the following contributions:

- We introduce **DemoEvolve**, a demonstration-bootstrapped approach to harness evolution. DemoEvolve augments the self-rollout archive with competent human trajectories and uses them as proposer-side reference experience for harness-level diagnosis and editing.
- We expose a noisy-selection failure mode of self-rollout harness evolution in long-horizon interactive tasks: same-seed rollout variance and compounding trajectory drift can make sparse scores favor inactive or noncausal harness edits.
- We provide Balatro evidence that demonstrations make sparse-feedback harness evolution more effective and auditable. Under the same limited budget, DemoEvolve produces active model-facing edits, improves performance over self-rollout and text-guided variants, and yields behavioral and diagnosis evidence consistent with better edit localization.

2 Related Work

2.1 Agent Harness Optimization and Experience-Based Self-Improvement

The performance of LLM agents is determined not only by model weights, but also by the external execution structures that govern prompting, tool use, memory, and multi-step orchestration. We use *harness* as an umbrella term for these task-specific structures, including prompts, workflows, tools,

memory, and execution logic. Recent work treats such structures as searchable or editable objects, optimizing prompts, workflows, tools, memories, or full agent harnesses through feedback-driven rewriting, program search, candidate selection, or outer-loop evolution [12, 42, 30, 24, 18, 15, 17, 28, 41, 34, 35]. A related line of work improves agents by converting interaction histories into reusable reflections, memories, or skills that can be retrieved in later tasks [31, 20, 44, 33, 39, 43, 40, 37, 19, 36]. These works typically assume that past trajectories, feedback, or execution logs contain improvement signals that can be extracted and reused. We instead ask when self-generated experience remains useful for harness evolution in non-model-native, long-horizon interactive tasks, where failures may be sparse, noisy, and hard to attribute to specific harness decisions.

2.2 Experience Distribution Shaping in Sequential Decision Making

In sequential decision-making, learning is strongly affected by the trajectories available early in training. Imitation learning and learning from demonstrations use expert behavior to provide reference trajectories, reduce exploration burden, and mitigate distribution mismatch between expert states and learner-induced states [29, 10, 9, 23, 27]. Curriculum and self-paced learning further show that controlling the order, difficulty, or goals of training experience can improve optimization stability and sample efficiency [1, 14, 6]. Related exploration methods for sparse-reward tasks also emphasize the importance of reaching informative or promising states rather than exploring uniformly from scratch [5]. Our work is related in spirit, but differs in that the LLM weights remain frozen and the optimized object is an external harness; demonstrations and opponent-conditioned trajectories serve as feedback for harness-level diagnosis and editing rather than direct policy training.

2.3 LLM Agents in Games and Dynamic Environments

Games provide a complementary testbed to static question answering and offline benchmarks. Recent benchmarks place language agents in text games, video games, board and card games, and multi-agent environments to evaluate dynamic state tracking, long-horizon planning, hidden-information reasoning, social reasoning, and opponent modeling [8, 25, 16, 11, 4, 26, 7]. Related work has also begun to use stochastic roguelike deck-building games to expose limitations of LLM agents in long-term resource management, combinatorial strategy, and risk control [3, 2]. Unlike work that primarily evaluates fixed agents or trains game-playing policies, we use games as test environments for harness self-evolution under stochasticity, long-horizon decisions, and dynamic interaction.

3 Method

This section formalizes demonstration-guided harness evolution in LLM-non-native task worlds. We first define the harness optimization problem around a frozen model, then distinguish simulator-level reproducibility from agent-facing uncertainty. We finally describe the outer-loop coding proposer and the three information regimes compared in our experiments.

3.1 Problem Formulation

Following Meta-Harness [15], we view harness evolution as end-to-end optimization over executable programs around a frozen model. Let W_T denote a task world, $x \sim \mathcal{X}_T$ an episode instance, π_ϕ a frozen model, and $h \in \mathcal{H}_T$ a task-oriented harness. The task world exposes observations and task-specific action interfaces. In games, these actions may include moves such as playing, discarding, buying, selling, or rerolling. The harness determines how observations are represented, which actions or tools are exposed, how model outputs are parsed, and how actions are executed in W_T . Together, W_T , π_ϕ , and h induce a rollout distribution over interaction trajectories. We optimize

$$h^* = \arg \max_{h \in \mathcal{H}_T} \mathbb{E}_{x \sim \mathcal{X}_T, \tau \sim p_{W_T, \pi_\phi, h}(\cdot | x)} [R_T(\tau, x)].$$

Here, $\tau = (o_0, a_0, o_1, a_1, \dots, o_T)$ is the task-world interaction trajectory and $R_T(\tau, x)$ is a sparse task reward, typically observed only after the rollout. The model parameters ϕ are fixed throughout; all improvement comes from changing the executable harness h .

Controlled task-world dynamics. Our goal is to study harness evolution in task worlds that are dynamic enough to require state abstraction and long-horizon action reasoning, but controlled enough

for systematic search and evaluation. We therefore distinguish the evaluator’s view of a fixed-seed simulator from the agent-facing view exposed through the harness. Let $\tilde{s}_t$ denote the complete task-world state, let $o_t = \Omega(\tilde{s}_t)$ be the observation shown to the agent, and let ρ be an episode seed. Under a fixed seed, the full-state transition may be deterministic, $\tilde{s}_{t+1} = F_\rho(\tilde{s}_t, a_t)$, making repeated evaluation on the same seed reproducible.

Reproducibility for the evaluator, however, does not imply that the agent knows the future consequences of its actions. The agent acts from its observation-action history $\eta_t = (o_0, a_0, \dots, o_t)$, not from the complete task-world state. The next observation therefore remains uncertain at the agent interface:

$$p_\rho(o_{t+1} \mid \eta_t, a_t) = \sum_{\tilde{s} \in \mathcal{S}} p_\rho(\tilde{s}_t = \tilde{s} \mid \eta_t) \mathbf{1}[o_{t+1} = \Omega(F_\rho(\tilde{s}, a_t))].$$

We use *agent-facing uncertainty* to refer to this induced uncertainty at the observation and decision interface, rather than to intrinsic nondeterminism after all simulator variables are fixed. Turn-based games therefore provide a controlled dynamic regime: they introduce action-conditioned future consequences and delayed feedback, while retaining fixed seeds and explicit action interfaces for reproducible harness evolution.

Program-level diagnostic credit assignment. The resulting optimization problem differs from standard game learning. In reinforcement learning, sparse outcomes are usually connected to policy or value updates through a predefined learning rule [32, 22]. In our setting, the optimized object is a non-differentiable executable harness: prompts, state tracking, action filters, tool exposure, and control flow. A terminal reward therefore provides no direct update target for a specific harness edit. The proposer must instead infer which harness mechanisms caused downstream success or failure from trajectories, execution logs, and sparse rewards.

This diagnostic problem is fragile when rollout scores are sparse and noisy. A terminal outcome may show that a run failed, but not whether the failure came from missing state abstractions, poor action filtering, weak prompt rendering, inappropriate tool exposure, or control-flow errors. Moreover, a candidate can appear better because of trajectory variance or seed-specific selection noise rather than because its edit is causally active. We therefore distinguish *score improvements* from *functional harness improvements*: the latter require the new harness logic to change the model-facing interface or execution behavior in an auditable way.

3.2 Harness Evolution under Different Information Regimes

We optimize h with an outer-loop proposer P . The proposer is a coding agent, not the task-execution agent itself. At each iteration, it inspects prior harnesses and their evaluation artifacts, diagnoses likely failure modes, and writes one or more candidate harnesses.

Following the Meta-Harness search loop [15], we maintain a growing filesystem archive $\mathcal{D}_t$. Each evaluated harness contributes one evaluation directory containing its source code, development-set scores, rollout trajectories, and raw execution traces. Rollout trajectories record the environment-level interaction τ , while raw execution traces record harness-level details such as contexts shown to the model, model outputs, parsed actions, tool calls, errors, and harness-side logs. The proposer queries this archive through normal file operations rather than receiving a compressed summary.

For notational compactness, let k_t be the number of candidates proposed at outer-loop iteration t . The update is

$$\{h^{(t+1,j)}\}_{j=1}^{k_t} \leftarrow P(I_t), \quad \mathcal{D}_{t+1} = \mathcal{D}_t \cup \{\text{EvalDir}(h^{(t+1,j)})\}_{j=1}^{k_t},$$

where I_t denotes the information available to the proposer, and $\text{EvalDir}(h^{(t+1,j)})$ is the evaluation directory written after running candidate $h^{(t+1,j)}$ on the development instances. Final test instances are held out and are not visible to the proposer during evolution.

We compare three information regimes:

$$I_t^{\text{meta}} = \mathcal{D}_t, \quad I_t^{\text{open}} = \mathcal{D}_t \cup K^{\text{text}}, \quad I_t^{\text{demo}} = \mathcal{D}_t \cup T^{\text{human}}.$$

Figure 2 summarizes the outer-loop process and the information available under each regime.

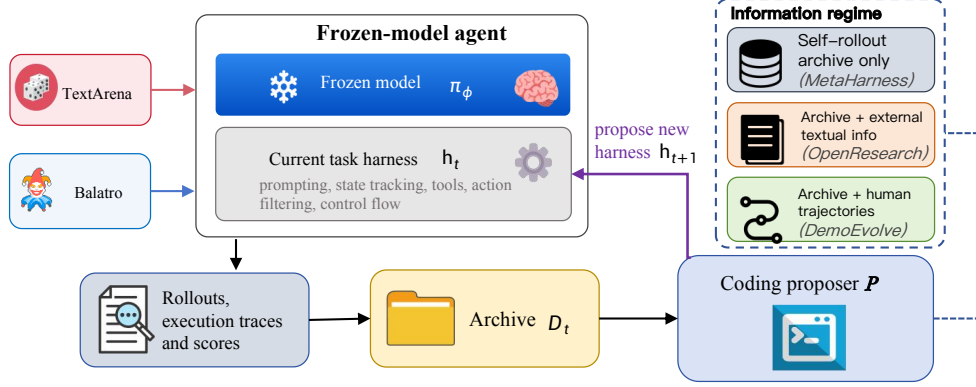


Figure 2: **DemoEvolve information regimes.** A coding proposer evolves a task harness around a frozen model using a filesystem archive of prior rollout trajectories, raw execution traces, and scores. We vary only the proposer-visible information: self-rollout archive, archive plus external textual information, or archive plus human trajectories.

Meta-Harness denotes the self-rollout evolution condition. In this regime, the proposer observes only the archive D_t : prior harness code, scores, self-generated rollout trajectories, and raw execution traces.

OpenResearch denotes a text-guided evolution condition. In addition to D_t , the proposer receives external textual task information K^{text} , such as rules, tutorials, strategy guides, or reference code. This condition tests whether high-level task knowledge alone is sufficient for long-horizon harness improvement.

DemoEvolve denotes our demonstration-guided evolution condition. In addition to D_t , the proposer receives competent human trajectories

$$T^{\text{human}} = \{(\tau_j^H, R_T(\tau_j^H, x_j))\}_{j=1}^m, \quad \tau_j^H = (o_0^H, a_0^H, o_1^H, a_1^H, \dots, o_{T_j}^H).$$

Human trajectories are represented in the same task-world observation-action format as agent rollouts, but their actions come from competent human play. We use these trajectories as behavioral exemplars and diagnostic references for the coding proposer, not as supervised action-level labels or model-finetuning data. Because they expose both competent actions and the concrete states in which those actions occur, they help the proposer compare failed agent rollouts against competent behavior and localize missing state abstractions, poor action priorities, weak resource management, or tools.

4 Experiments

We use games to test whether harness evolution can turn practice into executable task heuristics around a frozen model. This is a natural target: many game skills, for humans as well, are learned as rules of thumb—when to call, raise, discard, save, or reroll—rather than as exhaustive planning. TextArena Liar’s Dice [8] is the optimistic case: although stochastic and hidden-information, useful play can be captured by compact local heuristics, and self-rollout evolution can discover such scaffolding from its own rollouts. Balatro [3, 2] is the harder case: useful heuristics exist, but their value is phase- and state-dependent, so sparse final scores can reward the wrong harness edit. We therefore use Balatro to test whether human demonstrations make this search more diagnosable and localizable.

4.1 Liar’s Dice: Self-Rollout Discovers Compact Heuristics

Setup. We first evaluate harness evolution in TextArena Liar’s Dice [8], a lightweight interactive game with stochastic rolls, hidden information, opponent interaction, and sequential decisions. This environment serves as a positive control before the longer-horizon Balatro setting: when rollout feedback is sufficiently attributable, the proposer should be able to convert observed failures into executable harness structure around a frozen model.

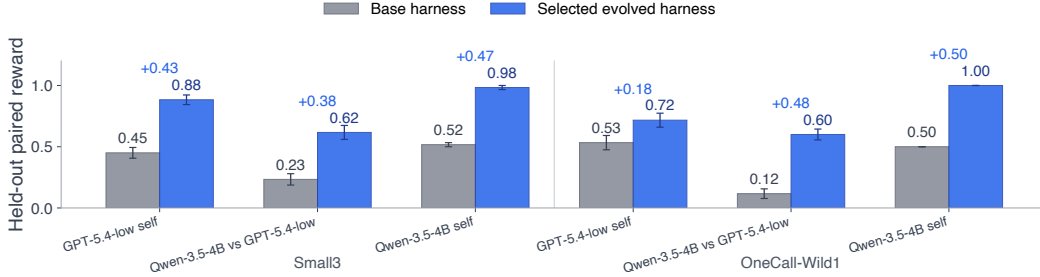


Figure 3: Held-out TextArena performance after development-only harness selection. Each bar uses 30 paired logical held-out seeds. Error bars show standard error. Bar labels show mean held-out reward, and the number above each pair is the evolved-minus-base reward difference. All six main comparisons improve on held-out seeds.

We use two Liar’s Dice variants. Small3 is a three-dice sequential bidding game, while OneCall-Wild1 is a five-dice single-call variant in which ones are wild. For each task, we evaluate three target/opponent cells: GPT-5.4-low self-play, Qwen-3.5-4B against GPT-5.4-low, and Qwen-3.5-4B self-play. Candidate harnesses are selected only from development/search rollouts. Held-out evaluation is run after selection on disjoint seeds, using paired-side control with 30 logical held-out seeds per comparison.

Results. The search loop finds higher-development-reward harnesses within a small number of proposals across both tasks and all model pairings; the full development curves are shown in Appendix A. The selected harnesses are then evaluated on held-out seeds. As shown in Figure 3, the task-balanced macro held-out reward improves from 0.392 for the base harness to 0.800 for the selected evolved harness, an absolute gain of +0.408. The improvement is consistent across tasks: Small3 improves from 0.400 to 0.828, and OneCall-Wild1 improves from 0.383 to 0.772. All six clean run-level comparisons improve on held-out seeds.

Mechanism. Inspection of the selected harnesses suggests that the gain is not merely from longer prompts. The evolved harnesses repeatedly add task-specific structure around the frozen model: observation parsing, legal-action guards, bid repair, probability estimates, expected-value call thresholds, opening policies, and conservative safety overrides.

Appendix B provides qualitative case studies of the selected Liar’s Dice harnesses. The cases show that harness evolution adapts the model–harness boundary to the base model and task variant: in one Qwen Small3 run, the selected harness becomes a near-symbolic threshold policy, while stronger-model and harder-variant runs retain a neuro-symbolic division of labor in which code handles legality, probability, and risk, and the model handles residual strategic judgment.

Interpretation. These results show that self-rollout evolution can turn practice into executable local heuristics around a frozen model. Liar’s Dice is the optimistic case for this pattern: useful play can be captured by compact rules of thumb, and rollout traces provide enough signal for the proposer to externalize brittle game-specific computation into deterministic harness. Balatro tests the limit of this pattern, where similar heuristics must be applied selectively across phases and states, and sparse final scores become a noisier guide for harness search.

4.2 Balatro: Sparse Feedback Can Mislead Long-Horizon Harness Search

Setup. Balatro is a poker deck-building roguelite [3, 2]. We use it as a long-horizon stochastic testbed because successful play requires delayed resource management under seed-dependent card draws, shops, boss blinds, and joker synergies. These properties make terminal feedback sparse, high-variance, and difficult to attribute to concrete harness mechanisms.

We use the fixed-seed setup from BalatroBench [2]. Seeds A/B/C are used as development seeds during evolution, while seeds D/E are held out and used only for final evaluation. Candidate harnesses are selected only from development-seed rollouts, and the selected harness is re-evaluated on all five

Table 1: Balatro results across in-distribution (ID) and out-of-distribution (OOD) seeds. Each cell reports capped mean final round, with completion rate in parentheses.

Method	ID seeds			OOD seeds		Averages		
	A	B	C	D	E	ID	OOD	Overall
BalatroLLM	12.0(0/3)	22.0(1/3)	17.0(2/3)	12.7(0/3)	21.0(2/3)	17.00(3/9)	16.83(2/6)	16.93(5/15)
Meta-Harness	16.7(0/3)	23.0(2/3)	18.3(2/3)	13.7(0/3)	19.0(2/3)	19.33(4/9)	16.33(2/6)	18.13(6/15)
OpenResearch	17.3(0/3)	15.0(1/3)	17.7(1/3)	14.0(0/3)	20.0(2/3)	16.67(2/9)	17.00(2/6)	16.80(4/15)
DemoEvolve	22.0(2/3)	24.0(3/3)	24.0(3/3)	16.0(1/3)	24.0(3/3)	23.33(8/9)	20.00(4/6)	22.00(12/15)

seeds. Full seed strings, rollout counts, selection rules, model settings, and cost details are provided in Appendix C.2.

We compare four conditions. **BalatroLLM** is the fixed BalatroLLM harness without outer-loop evolution [3]. **Meta-Harness** is the self-rollout evolution condition, where the coding proposer sees only prior harness code, scores, self-generated rollouts, and execution traces [15]. **OpenResearch** adds external textual task information such as rules, tutorials, strategy guides, or reference code. **DemoEvolve** adds competent human trajectories in the same observation-action format as agent rollouts, allowing the proposer to inspect concrete state-action evidence when proposing harness edits. Across evolved conditions, the initial harness, search budget, task model, proposer, selection rule, and final evaluation protocol are held fixed; the intended difference is only the information available to the proposer.

Results. Table 1 reports fixed-seed Balatro performance. BalatroLLM completes 5/15 rollouts. Meta-Harness, the self-rollout evolution condition, shows an apparent ID-seed gain, increasing ID mean final round from 17.00 to 19.33 and completion from 3/9 to 4/9. However, this gain does not transfer: OOD mean round changes from 16.83 to 16.33, with the same 2/6 completion count.

We therefore audit whether the selected harnesses make active model-facing changes. For each selected harness, we inspect the implementation and replay fixed-seed rollouts with additional logging, checking whether added hooks trigger, derived state variables are computed and injected, and model-call inputs or action interfaces change. The selected Meta-Harness candidate contains an intended dynamic hook, but the hook is never triggered because of an implementation error. Its apparent ID gain is therefore not a verified functional harness improvement, but is better interpreted as noisy candidate selection under high-variance rollout evaluation; Appendix C.3 provides the render-level audit.

OpenResearch, which adds external textual task information but no human trajectories, also fails to improve overall performance, completing 4/15 rollouts. By contrast, DemoEvolve improves both ID and OOD seeds: it completes 8/9 ID rollouts and 4/6 OOD rollouts, for 12/15 overall. In contrast, our audit indicates that the selected DemoEvolve changes are active and model-facing, introducing state- and phase-dependent information grounded in human trajectory evidence. These results suggest that the main Balatro issue is not only low score, but unreliable attribution: sparse self-rollout evolution can select inactive or noncausal edits, while human trajectories provide more diagnostic evidence for harness-level credit assignment.

4.3 Behavioral Analysis: Human Demonstrations Transfer Strategy

The final-performance results show that DemoEvolve performs better, but not what behavior changes. We therefore examine whether human trajectories transfer a strategy-level pattern of competent play. We focus on economy management, since Balatro success requires the agent to preserve money across shops while still converting resources into enough scoring power.

Exact action matching is not a reliable target in Balatro: the value of a reroll, pack purchase, discard, or played hand depends on the current joker set, hand levels, boss blind, deck composition, and shop offers. We instead use economy curves as a coarse but measurable behavioral probe of long-horizon strategy.

To avoid survivorship bias, we compute shop-entry money over all initial rollouts, assigning zero to rollouts that have already died. For method m , seed panel s , and shop round r , let $N_{m,s}$ be the

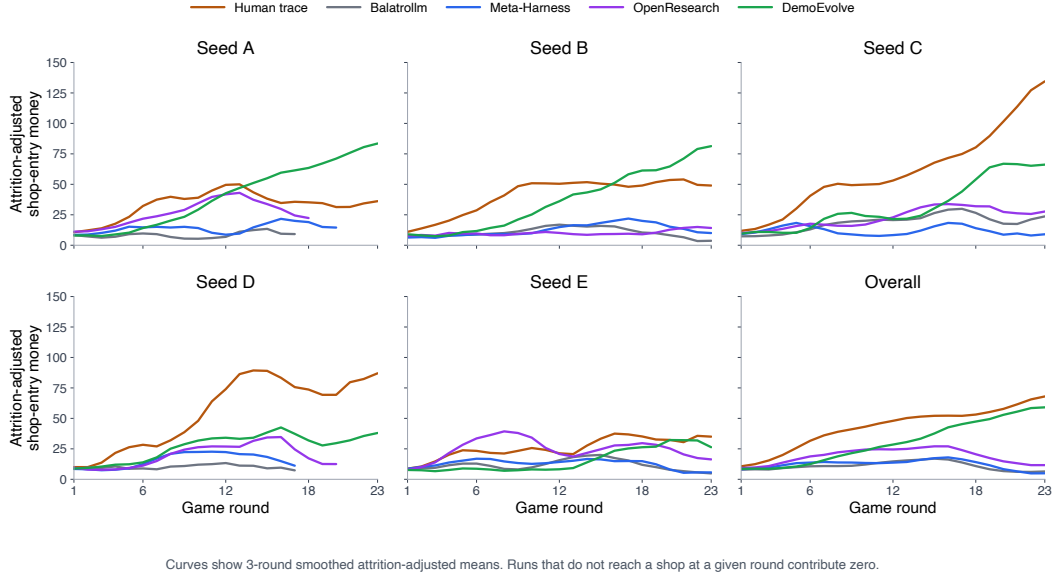


Figure 4: Attrition-adjusted shop-entry economy in Balatro. Dead rollouts contribute zero at later shops, so late values reflect both survival and money management. A/B/C are in-distribution seeds and D/E are held out; human traces on D/E are post-hoc references only and are never visible during evolution. The bottom-right panel averages over all seeds. Faint lines show raw adjusted means and bold lines show three-round moving averages.

Table 2: Summary of attrition-adjusted economy behavior. Entry metrics are computed from $\bar{M}_m(r)$; late metrics average rounds 19–23. Human distances are mean absolute differences from the human-trace adjusted economy curve, with lower values indicating closer behavior.

Method	Adj. entry mean	Late adj. entry	Late reach	Human dist.	Late human dist.
Human trajectories	42.99	61.87	1.00	0.00	0.00
BalatroLLM	10.89	6.61	0.39	32.10	55.25
Meta-Harness	12.16	6.97	0.45	30.83	54.89
OpenResearch	18.96	13.51	0.32	24.03	48.36
DemoEvolve	30.73	55.37	0.81	12.26	6.49

number of initial rollouts. We define adjusted shop-entry money as

$$\widetilde{M}_{m,s}(r) = \frac{1}{N_{m,s}} \sum_{i=1}^{N_{m,s}} \widehat{M}_i(r), \quad \widehat{M}_i(r) = \begin{cases} M_i(r), & \text{if rollout } i \text{ reaches shop } r, \\ 0, & \text{otherwise.} \end{cases}$$

Here $M_i(r)$ is the shop-entry money of rollout i at round r . Thus, late-round values reflect both survival and economy management. The plotted foreground curve applies a three-round moving average for readability; all quantitative values below are computed from the unsmoothed adjusted curve.

Table 2 shows the same pattern numerically. Meta-Harness barely changes the economy profile relative to BalatroLLM: the overall human distance decreases only from 32.10 to 30.83, and the late-game distance remains high. OpenResearch moves closer to the human curve overall (24.03), but its late reach is only 0.32, so the behavior is not reliably converted into survival.

DemoEvolve is qualitatively different. Its overall human-economy distance falls to 12.26, its late-game distance falls to 6.49, and its late reach remains 0.81. This supports a strategy-level transfer interpretation: human trajectories help the evolved harness support a human-like resource-management profile, rather than merely improving final scores through lucky harness selection.

4.4 Localization Analysis: Demonstrations Improve State-Conditioned Edit Localization

We isolate modification localization with a 5×3 diagnosis/design study, separate from the evolution runs. All arms inspect the same fixed BalatroLLM rollouts and differ only in extra evidence: none, human notes, or human trajectories. For each arm, five independent proposer replicates diagnose the failures and propose concrete edit sites.

We classify edits by context-injection granularity. Pers. denotes persistent context; Phase denotes middleware gated only by the environment-provided phase field; State denotes harness-triggered computation of derived decision variables before injection; and Tool denotes model-invoked computation over the current state. We report Comp. rate = (State + Tool)/total, the fraction of edits that provide executable decision support. State and Tool differ in trigger: State is injected by the harness, while Tool delegates invocation to the model.

Table 3 separates defect localization from correction signal. Rollouts alone localize failure-visible defects, explaining their high Comp. rate (46.2%), but the evidence is negative-only: it shows where the agent broke, not what competent behavior would have done. Notes supply improvement principles, but without observation-action alignment they mostly become persistent or phase-level reminders, yielding a lower Comp. rate (21.7%). Human trajectories provide positive state-action counterfactuals, restoring a comparable Comp. rate (45.5%) while grounding computed interventions in observed conditions for when they should be injected or invoked.

Table 3: Context-injection granularity in the 5×3 diagnosis/design study. Entries are proposed edit sites. All rows inspect the same BalatroLLM rollouts; extra ev. denotes additional evidence. Comp. rate counts State and Tool edits.

Extra ev.	Pers.	Phase	State	Tool	Comp. rate
None	7/26	7/26	10/26	2/26	46.2%
Notes	9/23	9/23	1/23	4/23	21.7%
Human trajectories	8/22	4/22	6/22	4/22	45.5%

Economy management illustrates the distinction. Rollout-only diagnoses see after-the-fact symptoms such as overspending, lost interest bonuses, or weak later-round buffers, so their edits become purchase-side safeguards. Human notes are not missing the principle: a notes-only replicate proposed a hand-selection microhint that unused hands become end-round income. The missing signal is the state boundary. Human trajectories show that early in a run, before any hand has been played, with multiple discards left and no strong scoring hand, competent players often discard first and then clear the round in fewer hands. This turns the principle into a state-computing intervention such as `round1_dig_hint`, gated by `hands_played=0`, `discards_left  $\geq$  2`, and early ante. Thus rollouts locate value loss, notes identify the principle, and human trajectories specify when the harness intervention should fire.

5 Conclusion

We studied harness evolution for frozen-model agents in long-horizon, stochastic interactive tasks, where sparse terminal feedback and high rollout variance make self-improvement difficult to diagnose. TextArena shows that self-rollout evolution can work when failures are short-horizon and attributable, while Balatro exposes a harder regime where sparse scores can select inactive or noncausal edits. DemoEvolve shows that competent demonstrations can make this search problem easier: they provide positive state-action evidence that narrows the effective harness search space and helps translate sparse failures into actionable, model-facing mechanisms. These results suggest that demonstrations can make sparse-feedback harness evolution more sample-efficient, diagnosable, and stable.

6 Limitations

Our main limitation is task coverage. DemoEvolve is evaluated primarily in Balatro, with TextArena serving as a lightweight positive control for self-rollout evolution under more attributable feedback. Although Balatro is a useful long-horizon stochastic testbed, it is still one game environment. Future work should test whether demonstration-bootstrapped harness evolution transfers to a broader range of interactive settings, including other games, tool-use agents, terminal tasks, web agents, and embodied environments.

References

- [1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48. Association for Computing Machinery, 2009.
- [2] Coder. Balatrobench. <https://balatrobench.com/>, 2026. Accessed May 2, 2026.
- [3] Coder. BalatroLLM: Play Balatro with LLMs. <https://github.com/coder/balatrollm>, 2026. Accessed May 2, 2026.
- [4] Jinhao Duan, Renming Zhang, James Diffenderfer, Bhavya Kailkhura, Lichao Sun, Elias Stengel-Eskin, Mohit Bansal, Tianlong Chen, and Kaidi Xu. Gtbench: Uncovering the strategic reasoning limitations of llms via game-theoretic evaluations, 2024. URL <https://arxiv.org/abs/2402.12348>.
- [5] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.
- [6] Carlos Florensa, David Held, Markus Wulfmeier, Michael Zhang, and Pieter Abbeel. Reverse curriculum generation for reinforcement learning. In *Conference on Robot Learning*, pages 482–495, 2017.
- [7] Lingyue Fu, Xin Ding, Linyue Pan, Yaoming Zhu, Shao Zhang, Lin Qiu, Weiwen Liu, Weinan Zhang, Xuezhi Cao, Xunliang Cai, Jiaxin Ding, and Yong Yu. Catarena: Evaluating evolutionary capabilities of code agents via iterative tournaments, 2025. URL <https://arxiv.org/abs/2510.26852>.
- [8] Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. Textarena, 2025. URL <https://arxiv.org/abs/2504.11442>.
- [9] Todd Hester, Mel Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Jean-Baptiste Lespiau, Laurent Sartran, and Guillaume Beaudoin. Deep q-learning from demonstrations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018.
- [10] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems*, 2016.
- [11] Lanxiang Hu, Qiyu Li, Anze Xie, Nan Jiang, Ion Stoica, Haojian Jin, and Hao Zhang. Gamearena: Evaluating llm reasoning through live computer games, 2024. URL <https://arxiv.org/abs/2412.06394>.
- [12] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=t9U3LW7JVX>.
- [13] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, October 2023. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- [14] M. Pawan Kumar, Benjamin Packer, and Daphne Koller. Self-paced learning for latent variable models. *Advances in Neural Information Processing Systems*, 23, 2010.
- [15] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, March 2026. URL <https://arxiv.org/abs/2603.28052>.
- [16] Wenhao Li, Wenwu Li, Chuyun Shen, Junjie Sheng, Zixiao Huang, Di Wu, Yun Hua, Wei Yin, Xiangfeng Wang, Hongyuan Zha, and Bo Jin. Textatari: 100k frames game playing with language agents, 2025. URL <https://arxiv.org/abs/2506.04098>.
- [17] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shiha Dou, Xuanjing Huang, Hang Yan, Zhenhua Han, and Tao Gui. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, April 2026. URL <https://arxiv.org/abs/2604.25850>.

- [18] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness, 2026. URL <https://arxiv.org/abs/2603.03329>.
- [19] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver, April 2026. URL <http://arxiv.org/abs/2604.08377>.
- [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL <https://openreview.net/forum?id=S37h0erQLB>.
- [21] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, January 2026. URL <http://arxiv.org/abs/2601.11868>.
- [22] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- [23] Ashvin Nair, Bob McGrew, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Overcoming exploration in reinforcement learning with demonstrations. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6292–6299, 2018.
- [24] Alexander Novikov, Ngán Vű, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, June 2025. URL <http://arxiv.org/abs/2506.13131>.
- [25] Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Lukasz Kucinski, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktaschel. Balrog: Benchmarking agentic llm and vlm reasoning on games, 2024. URL <https://arxiv.org/abs/2411.13543>.
- [26] Dongmin Park, Minkyu Kim, Beongjun Choi, Junhyuck Kim, Keon Lee, Jonghyun Lee, Inkyu Park, Byeong-Uk Lee, Jaeyoung Hwang, Jaewoo Ahn, Ameya S. Mahabaleshwar, Bilal Kartal, Pritam Biswas, Yoshi Suhara, Kangwook Lee, and Jaewoong Cho. Orak: A foundational benchmark for training and evaluating llm agents on diverse video games, 2025. URL <https://arxiv.org/abs/2506.03610>.
- [27] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. In *Robotics: Science and Systems*, 2018.

- [28] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL <https://arxiv.org/abs/2504.15228>.
- [29] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.
- [30] Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent. <https://github.com/algorithmicsuperintelligence/openevolve>, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>. GitHub repository.
- [31] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Thirty-Seventh Conference on Neural Information Processing Systems*, November 2023. URL <https://openreview.net/forum?id=vAElhFcKW6>.
- [32] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [33] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, October 2023. URL <http://arxiv.org/abs/2305.16291>.
- [34] Wenyi Wang, Piotr Piekos, Nanbo Li, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jurgen Schmidhuber. Huxley-godel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025. URL <https://arxiv.org/abs/2510.21614>.
- [35] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly?, 2025. URL <https://arxiv.org/abs/2511.13646>.
- [36] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, February 2026. URL <http://arxiv.org/abs/2602.08234>.
- [37] Yiming Xiong, Shengran Hu, and Jeff Clune. Learning to continually learn via meta-learning agentic memory designs. In *OpenReview*, 2026. URL <https://api.semanticscholar.org/CorpusID:285454009>.
- [38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [39] Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution. *arXiv preprint arXiv:2601.21557*, 2026.
- [40] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*, 2025.
- [41] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents, 2026. URL <https://arxiv.org/abs/2505.22954>.
- [42] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, October 2024. URL <https://openreview.net/forum?id=z5uVAKwmjf>.

- [43] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*, October 2025. URL <https://openreview.net/forum?id=eC4ygDs02R>.
- [44] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, December 2024. URL <http://arxiv.org/abs/2308.10144>.

A TextArena Liar’s Dice Details

This appendix gives additional details for the TextArena Liar’s Dice experiments reported in Section 4.1. These experiments serve as a lightweight positive control: they test whether self-rollout harness evolution works when feedback is relatively short-horizon and attributable, before moving to the longer-horizon Balatro setting.

Game variants. We report two two-player Liar’s Dice variants. **Small3** uses the `LiarsDice-v0-small` environment: each player starts with three dice, ones are not wild, players alternately bid a quantity/face pair or call the previous bid, and losing a challenge removes one die. **OneCall-Wild1** uses the `LiarsDice-v0-onecall` environment: each player starts with five dice, ones are wild until face 1 is bid, and the first `[Call]` immediately ends the match without dice loss or rerolls. These variants are interactive and stochastic, with hidden dice, opponent pressure, legal-action constraints, and sequential bidding decisions. However, compared with Balatro, each episode is short enough that terminal outcomes can usually be traced back to a small number of bid or call decisions. They therefore provide a cleaner setting for testing whether self-generated rollout traces can support harness edits.

Role as a lightweight positive control. We use TextArena Liar’s Dice to check that the outer-loop machinery can improve a frozen-model agent when failures are attributable and the search signal is relatively dense. The point of this experiment is not to show that self-rollout feedback is sufficient for sparse, long-horizon games. Instead, it establishes a positive case: when the task horizon is short and rollout failures are easy to localize, self-rollout harness evolution can convert observed failures into useful executable harness structure.

Search and selection. For each target/opponent/task cell, the outer-loop proposer modifies only the external harness around a frozen target model. Candidate selection uses development/search rollouts only; held-out seeds are not visible during candidate proposal or selection. The search budget is deliberately small: candidate evaluations usually use $N = 5$ logical development seeds, while some later Small3 candidates and baselines use $N = 10$. The selected harness is chosen according to development reward only. One OneCall-Wild1 GPT self-play run had a pre-declared development-only tie-break: two candidates both reached development reward 0.700, and the clean tie-break selected the later candidate. This tie-break did not use held-out performance.

Held-out paired evaluation. Final evaluation uses 30 matched logical held-out seeds for every base/evolved comparison. Small3 uses held-out logical seeds 120000–120029, and OneCall-Wild1 uses held-out logical seeds 130000–130029. The same seed set is used for the base harness and the selected evolved harness within each cell.

We run held-out evaluation with paired-side control. Each logical seed is evaluated as two raw games with the same environment seed and swapped player slots. The reported paired reward averages the two legs, so a win receives reward 1, a draw receives reward 0.5, and a loss receives reward 0. Thus $N = 30$ means 30 paired logical matches, or 60 raw games, for each base or evolved row. This controls for first-player and seat-order effects while keeping the stochastic dice rolls fixed across the base and evolved comparisons.

Metrics and clean-run criteria. The primary metric is mean held-out paired reward over the 30 logical seeds, with standard errors computed across paired rewards. The paper-level aggregate is task-balanced: we average run-level means within Small3 and OneCall-Wild1, then average the two task means. In the clean tie-break package, this gives 0.392 for the base harness and 0.800 for the selected evolved harness, an absolute gain of +0.408.

A row is included in the clean main result only if all 30 base and 30 evolved paired summaries are present, paired-side evaluation succeeds for all logical seeds, and there are no LLM fallback, error, or timeout events on either target or opponent side. Invalid actions and parse fallbacks are logged as behavioral audit metrics rather than infrastructure failures.

Artifact regeneration. The reported TextArena numbers are generated from archived clean tie-break artifacts containing raw held-out traces, held-out summaries, macro summaries, audit files,

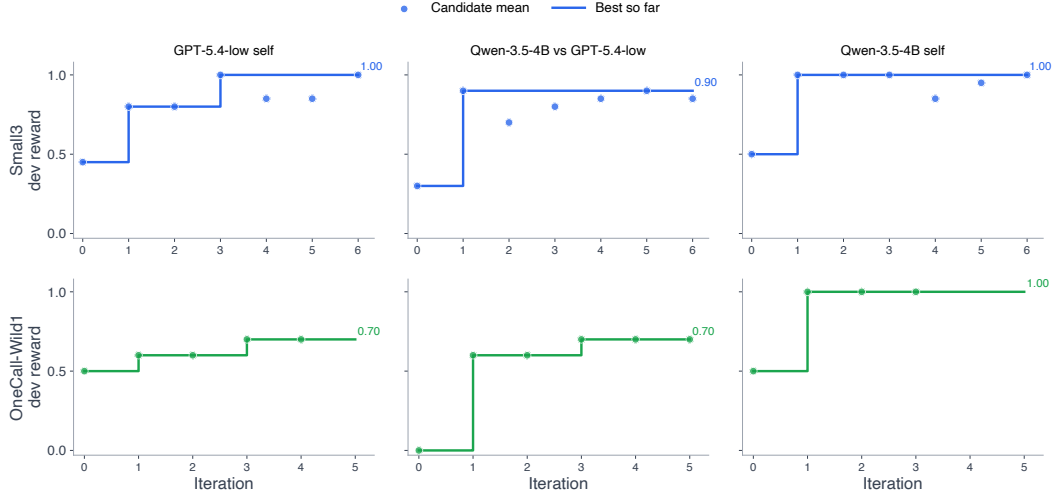


Figure 5: Development/search progress for TextArena Liar’s Dice. Iteration 0 is the base harness. Points show candidate development reward, and lines show the best development reward observed so far. Candidate selection uses only development/search rollouts; held-out seeds are not used during search.

source snapshots, and per-candidate search-progress records. Execution parallelism only controls concurrency and does not repeat evaluation on the same seed. As with any evaluation using hosted or locally served LLMs, exact bit-for-bit replay can depend on model-serving implementation, endpoint availability, and retry behavior. The reported numbers are therefore tied to the archived raw traces and summaries, while the seeded protocol specifies the stochastic game instances used for reproduction.

B Qualitative Analysis of Evolved Liar’s Dice Harnesses

This section qualitatively inspects selected Liar’s Dice harnesses from the TextArena experiments in Section 4.1. The goal is not to present the full source code, but to understand what kinds of executable heuristics self-rollout harness evolution discovered. Across the selected harnesses, we observe a recurring pattern: the outer loop moves brittle decision computations out of free-form language-model reasoning and into executable scaffolding. Depending on the base model and game variant, this scaffolding ranges from near-symbolic control to a neuro-symbolic division of labor.

Case A: Near-symbolic threshold control for a weaker model. In the Small3 Qwen self-play setting, the selected harness largely bypasses the language model at decision time. It parses the observation, estimates whether the current bid is true, calls when the bid appears sufficiently unlikely, and otherwise raises to a minimally legal bid with acceptable probability. A simplified sketch is:

```
state = parse_observation(observation)

if no_current_bid:
    return opening_bid_from_own_dice(state)

p_true = P(current_bid_is_true | own_dice, unknown_opponent_dice)

if p_true < 0.30:
    return "[Call]"

raise = smallest_legal_raise_with_probability_at_least(0.50)
return raise
```

This case is useful precisely because it is close to a limiting case. Harness evolution is not constrained to preserve a fixed role for the language model. When the base model is weak or unstable for the

local game mechanics, the search can improve the overall frozen-model agent by shifting more of the decision procedure into deterministic code. In this lightweight setting, the resulting “skill” resembles a compact rule-of-thumb policy rather than improved model reasoning: the optimized object is the full model–harness system.

Case B: Opponent-aware expected-value scoring. In the Small3 GPT-5.4 self-play setting, the selected harness keeps a more hybrid structure. The harness parses the accumulated observation history, constructs a simple opponent face prior from the opponent’s bids in the current round, computes binomial-tail probabilities for the current claim and candidate raises, and ranks legal actions by expected value. A simplified sketch is:

```

opp_prior = compute_opp_face_prior(opponent_bids_this_round)

p_current = P(current_bid_true | own_dice, opp_dice_count, opp_prior)
ev_call = 2 * (1 - p_current) - 1

for raise in legal_raises(current_bid):
    p_raise = P(raise_true | own_dice, opp_dice_count, opp_prior)
    ev_raise = 2 * p_raise - 1

return best_action_by_ev([call] + legal_raises)

```

This harness does not merely decide what text to show the model. It computes decision-relevant variables that the model is brittle at maintaining across turns: legal raises, claim probabilities, opponent-conditioned priors, and call-versus-raise tradeoffs. The language model can still be used for residual strategic judgment or action verbalization, but the fragile arithmetic and legality constraints are stabilized outside the model.

Case C: Bayesian pile-on guard in OneCall-Wild1. OneCall-Wild1 is a harder Liar’s Dice variant because ones are wild and a single call ends the match. In the selected GPT-5.4 harness, the evolved logic adds a state-conditioned guard for deep same-face escalation. The harness estimates the truth probability of the current bid, estimates the best follow-up raise under a Bayesian pile-on model conditioned on the opponent’s last bid, and overrides continued raising when calling has higher expected value by a margin:

```

p_truth = P(current_bid_true | own_dice, wild_rule)
best_raise = argmax_raise P(raise_true | opponent_last_bid, wild_rule)

ev_call = 1.0 - p_truth
ev_raise = P(best_raise_true | bayesian_pile_on_model)

if own_contribution >= 2 and ev_call > ev_raise + margin:
    action = "[Call]"
else:
    action = best_raise

```

This case illustrates a more explicitly neuro-symbolic pattern. The harness handles rule interpretation, probability estimation, legal-action validation, and a high-risk override, while still allowing model-facing analysis or recommendations in ambiguous states. The intervention is state-conditioned rather than seed-conditioned: it fires on a structural pattern in the bidding trajectory rather than on a particular development seed.

Takeaway. The Liar’s Dice case studies show that self-rollout harness evolution can discover compact executable game heuristics. In easier or weaker-model settings, the selected harness may become nearly symbolic, replacing the model’s unstable local decisions with deterministic thresholds and guards. In harder variants or with stronger models, the harness tends to form a hybrid system: symbolic code handles explicit rules, probabilities, legal actions, and safety constraints, while the frozen language model handles residual semantic and strategic judgment. This supports the view that harness evolution optimizes the behavior of the entire model–harness system, not merely the prompt or the model’s standalone generation quality.

C Balatro Background and Terminology

C.1 Game Rules and Terminology

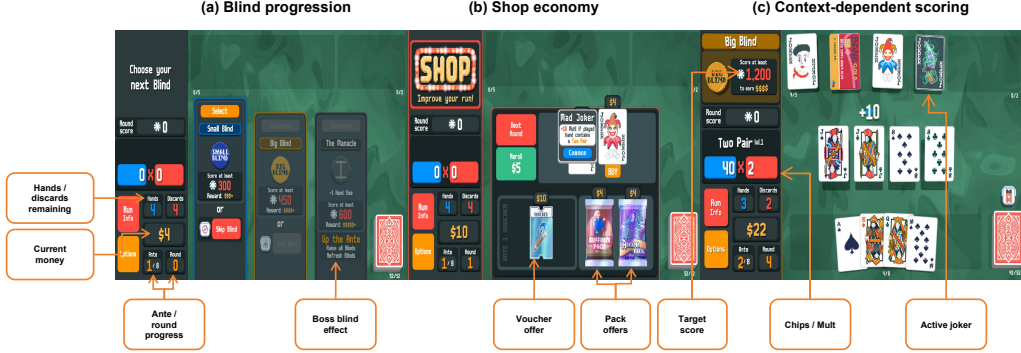


Figure 6: Annotated Balatro screenshots illustrating the mechanics used in our analysis. **(a)** Blind progression defines the survival objective: the agent must clear increasing target scores with limited hands and discards, while boss blinds add special constraints. **(b)** The shop phase turns money into a long-horizon resource, since the agent must choose among purchases, rerolls, and saving for future shops. **(c)** Scoring is build-dependent: the same played hand can have different value depending on chips, multipliers, remaining resources, and active jokers.

This section provides a compact introduction to Balatro for readers who have not played the game. We focus on the mechanics needed to interpret our benchmark, metrics, and behavioral analyses.

Run structure. A Balatro run consists of a sequence of *antes*, each containing several *blinds*. To clear a blind, the player must reach a target score before exhausting the available hands and discards. Failure to reach the target ends the run. We therefore use a rollout’s *final round* as a coarse measure of survival depth.

Scoring and hand play. During each blind, the player plays poker-style hands. The score of a played hand depends on the hand category, card values, hand levels, multipliers, and active modifiers. As a result, the best action is often not the strongest conventional poker hand in isolation; it depends on the current build, available scaling effects, and remaining resources.

Shops and economy. After blinds, the player enters a shop phase and can spend money on jokers, vouchers, booster packs, consumables, or rerolls. Money is therefore a strategic resource rather than a purely local reward signal. Spending aggressively may improve immediate survival, but preserving money can enable stronger future shops and more flexible scaling. This motivates our analysis of *shop-entry money* as a long-horizon behavioral probe.

Jokers and synergies. Jokers are persistent modifiers that often define a run’s strategy. Their value is highly compositional: a joker may be weak on its own, strong only with particular deck edits or consumables, or useful early but less valuable later. This context dependence makes action values difficult to interpret without considering the current joker set, deck composition, and stage of the run.

Randomness and seeds. Balatro contains substantial stochasticity, including shop inventories, pack contents, boss blind effects, and card draws. Fixing a seed makes a run reproducible, but repeated optimization on a small set of seeds can still exploit seed-specific opportunities rather than learn generally useful strategies. We therefore distinguish between development seeds used during harness evolution and held-out evaluation seeds used only for final evaluation.

Terminology used in the main text. The main paper uses several Balatro-specific terms:

- **Blind:** a scoring challenge that must be cleared to continue the run.

- **Ante:** a stage of the run containing multiple blinds; later antes generally require higher scores.
- **Shop-entry money:** the amount of money available when entering a shop after a blind.
- **Joker set:** the current collection of jokers shaping the run’s scoring, economy, and scaling behavior.
- **Boss blind:** a blind with an additional constraint or effect that can disrupt otherwise strong tactics.
- **Reroll:** spending money to refresh the shop’s offerings.

C.2 Experimental Protocol

Fixed seeds and split. We use the fixed-seed setup from BalatroBench [2]. Each selected harness is evaluated on five deterministic Balatro seeds, with three independent rollouts per seed. For readability, we abbreviate the concrete seed strings AAAAAA–EEEEEE as A–E. Seeds A/B/C are used as development seeds during evolution, while seeds D/E are held out and used only for final evaluation. The proposer never observes rollouts, scores, or traces from D/E during evolution.

Search and selection. For each evolution condition, we start from the same BalatroLLM harness [3] and run a 10-iteration outer loop. In each iteration, the coding proposer typically produces two candidate harnesses. Each candidate is evaluated on seeds A/B/C with one rollout per seed during search. Following the Meta-Harness protocol [15], the proposer may inspect the archive of previous harnesses, scores, self-generated rollouts, and execution traces, and may choose which prior version to build on. After the search budget is exhausted, we select the candidate with the highest mean final round on A/B/C, using completion rate on A/B/C as the tie-breaker when necessary. The selected candidate is then re-evaluated from scratch on all five seeds A–E with three independent rollouts per seed.

Rationale for the fixed-seed low-sample protocol. BalatroBench was originally designed as a fixed-seed model-evaluation setting rather than as a harness-evolution benchmark. We adopt fixed seeds because they make candidate comparisons diagnosable during outer-loop search: if each candidate were evaluated on different game instances, score changes would conflate harness edits with seed-level variation. At the same time, evaluating every candidate on many seeds and rollouts would make each evolution step slow and expensive, since every candidate requires full long-horizon LLM-agent rollouts. Our protocol therefore uses a low-sample development budget during search and a fixed-seed re-evaluation of the selected candidate. This is an engineering trade-off rather than a claim that the resulting scores are variance-free estimates of average Balatro skill. We interpret the fixed-seed results together with held-out-seed transfer, functional audits, and behavioral analyses.

Condition-specific inputs. **BalatroLLM** is the fixed BalatroLLM harness without outer-loop evolution and is evaluated only under the final evaluation protocol. **Meta-Harness** exposes only the archive of prior harness code, development-seed scores, self-generated rollouts, and raw execution traces. **OpenResearch** additionally gives the proposer web-search access and encourages it to search for external task information, such as rules, tutorials, strategy guides, or reference code, but provides no human trajectories. **DemoEvolve** provides nine competent human trajectories, consisting of three trajectories for each development seed A/B/C. No human trajectories are provided for held-out seeds D/E.

Models and controls. All Balatro task-execution agents use GPT-5.4-low as the frozen model, and all evolution conditions use Claude Opus 4.7 Max as the coding proposer. Across Meta-Harness, OpenResearch, and DemoEvolve, the initial harness, search budget, proposer model, candidate-selection rule, and final evaluation protocol are held fixed.

Compute and API cost. We estimate the Balatro rollout cost using GPT-5.4-low pricing. The final fixed-seed evaluation uses five seeds with three rollouts per seed, i.e., 15 rollouts in total, and costs approximately \$32, or about \$2.13 per rollout. A typical evolution run uses 10 iterations, with about two candidate harnesses per iteration; each candidate is evaluated on three development seeds. This gives $10 \times 2 \times 3 = 60$ development rollouts, corresponding to approximately \$128. Including the

final fixed-seed evaluation of the selected harness, the total rollout cost is about \$160 per evolution run. This estimate only includes task-execution rollouts and excludes the cost of the coding proposer.

C.3 Functional Audit of the Selected Candidate

This appendix audits the final candidate selected by the Meta-Harness self-rollout baseline as its best Balatro harness under the A/B/C development-seed selection rule. The selected candidate was `face_exposure_shop_hook`. It proposed a SHOP-phase hook for face-card-dependent builds, intended to warn the agent before *The Plant*, a boss blind that debuffs all face cards. The source contained a guarded prompt block:

```
{ % if hooks.face_exposure % } ... Face-Card Exposure Metrics ...
```

and candidate selection favored this version on the A/B/C development rollouts.

We audited the rendered model requests to test whether the proposed mechanism actually reached the model. Across the three A/B/C validation rollouts, we inspected 517 requests in `requests.jsonl`. None of the distinctive hook strings appeared:

Expected hook string	Occurrences
Face-Card Exposure Metrics	0
Deck face cards	0
face-dependent jokers	0
The Plant boss disables	0

Thus, although the source diff was non-empty, the advertised face-exposure intervention was absent from the model-facing context. The candidate’s development-set gain therefore cannot be attributed to the proposed hook.

This illustrates the noisy-selection failure mode in Balatro. Even with the same fixed seed and nearly identical prompts, a frontier LLM can produce different early actions; a single shop purchase, discard, or played hand can change the future state distribution and final outcome. With only a few long rollouts per candidate during search, an inert edit can be selected when ordinary rollout variance makes its score appear better than the reference. This is a credit-assignment failure: the optimizer observes a higher sparse reward and attributes it to the candidate diff, while the executed-context audit shows that the claimed mechanism was inactive.

For long-horizon harness evolution, source-level diffs are therefore insufficient. Selected edits should be audited at the rendered-context or execution level to verify that the intended prompt block, hook output, memory entry, or tool schema is active at the decision states where it is supposed to apply.